SYNOPSIS

PROJECT TITLE

SmartRoad: Image-Based Road Condition Monitoring, Mapping and Repair Prioritization System

1. INTRODUCTION

Roads are an essential part of transportation infrastructure, but their condition gradually deteriorates because of heavy traffic, weather conditions, water accumulation, poor maintenance and regular wear and tear. Common problems such as potholes, cracks, damaged edges and uneven road surfaces can reduce driving comfort, damage vehicles and create safety risks.

Traditional road-condition monitoring mainly depends on manual inspection. Inspectors have to physically survey roads, identify defects and record their locations. This process can be time-consuming and difficult to repeat frequently over a large area. Sensor-based monitoring systems are another approach, but they may require additional hardware, installation and maintenance.

The proposed SmartRoad system uses photographs of roads instead of dedicated physical sensors. A photograph captured using a mobile phone, camera or an existing road-image dataset is provided as input. The system processes the image, identifies visible road defects, estimates their severity and associates the defect with a particular road/location.

The system further provides a 2D map-based view of a selected area in which road conditions can be marked using different indicators. Green, yellow and red markers can represent good, moderate and poor road conditions respectively. This makes the condition of a road network easy to understand visually.

The project combines image processing with Data Structures and Algorithms in C, Object-Oriented Programming in C++, graph algorithms and GUI development to create a practical academic prototype.

2. PROBLEM STATEMENT

There is a need for a simple and cost-effective road-condition monitoring system that can use road photographs to identify visible defects and represent their locations on a map.

The system should be able to:

- Accept road photographs as input.

- Detect visible defects such as potholes and cracks.

- Classify defects according to type.

- Estimate the severity of each defect.

- Associate defects with road locations.

- Display road conditions on a 2D map.

- Store and manage defect information efficiently.

- Prioritize repair work according to severity.

- Suggest a suitable maintenance route.

3. MOTIVATION

The main motivation of this project is to make road-condition monitoring more accessible by using photographs rather than dedicated sensor hardware.

A road photograph contains visual information about the road surface. Image-processing techniques can be used to analyze this information and identify damaged regions. The detected information can then be converted into structured records and displayed on a map.

The project also provides a practical application of academic concepts. Queue, Linked List, Priority Queue and Graph are applied to actual system requirements. Similarly, OOP concepts such as abstraction, encapsulation, inheritance and polymorphism are used to model different road-defect types.

The map-based interface further improves the system because users can select a particular area and visually identify which roads require attention.

4. OBJECTIVES

- To develop an image-based road-condition monitoring system.

- To use road photographs as the primary input instead of dedicated physical sensors.

- To detect visible road defects such as potholes, cracks and damaged regions.

- To classify detected defects according to their type.

- To estimate defect severity as Low, Medium or High.

- To associate detected defects with a particular road or location.

- To display road conditions and defect markers on a 2D map.

- To implement Queue, Linked List and Priority Queue for efficient data management.

- To represent the road network using a Graph.

- To use Dijkstra's algorithm or A* for maintenance-route planning.

- To apply OOP concepts using C++.

- To provide a simple and user-friendly GUI dashboard.

5. PROPOSED SYSTEM

The SmartRoad system will work as an integrated image-analysis and map-based road-monitoring application.

First, the user selects a road photograph. The image is passed to the image-processing module, where basic preprocessing such as resizing, grayscale conversion, noise reduction, contrast enhancement and edge/region analysis may be performed.

The processed image is then analyzed to identify candidate damaged regions. Depending on the available dataset and implementation level, defect detection may be performed using image-processing rules, thresholding, contours/features or a lightweight trained model.

After detection, a RoadDefect object is created. Different defect types such as Pothole, Crack and DamagedEdge can be represented as derived classes of a common RoadDefect class.

Each defect can contain information such as:

- Defect ID

- Defect type

- Severity

- Road name/ID

- Approximate location or coordinates

- Image reference

- Repair status

The defect is then added to the appropriate DSA structure. A Queue can manage incoming images, a Linked List can maintain defect history, and a Priority Queue can arrange repair tasks according to severity.

The road network is represented as a weighted Graph. Intersections are treated as nodes and road segments as edges. The system can use Dijkstra's algorithm or A* to calculate a suitable maintenance route between the depot and selected defect locations.

Finally, the GUI displays an OpenStreetMap-based map, road-condition markers, defect information, repair priority and suggested route. Leaflet can be used as the map interface layer to display OpenStreetMap tiles and interactive markers.

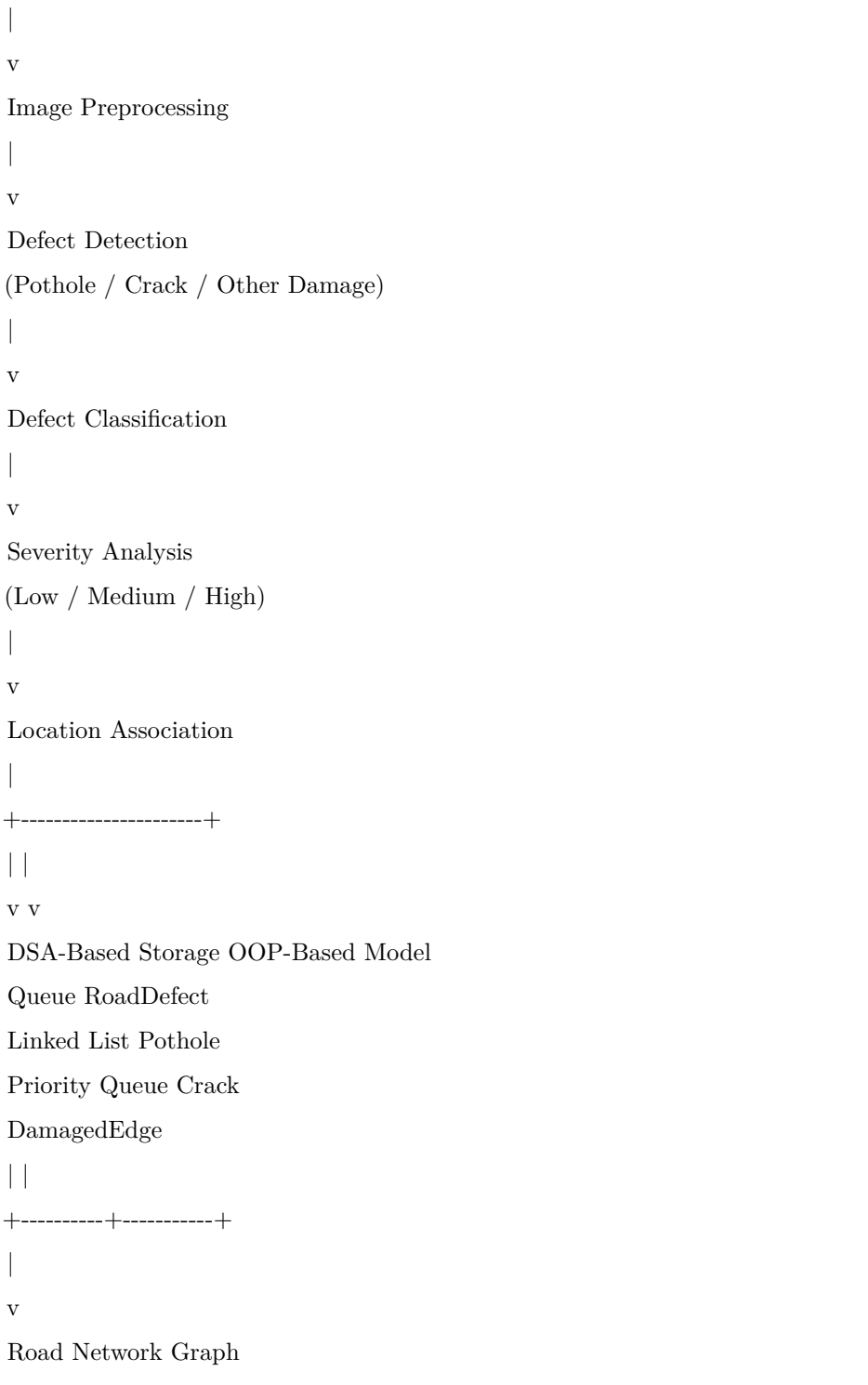
6. SYSTEM WORKFLOW

Road Photograph

|

v

Image Input


|
v
Image Preprocessing
|
v
Defect Detection
(Pothole / Crack / Other Damage)
|
v
Defect Classification
|
v
Severity Analysis
(Low / Medium / High)
|
v
Location Association
|
+----------+
| |
v v
DSA-Based Storage OOP-Based Model
Queue RoadDefect
Linked List Pothole
Priority Queue Crack
DamagedEdge
| |
+----------+----------+
|
v
Road Network Graph

|

v

Dijkstra / A*

|

v

Maintenance Route

|

v

GUI Dashboard

Map + Defects + Priority + Route

7. MAP-BASED ROAD CONDITION MONITORING

One of the main features of the proposed system is a 2D map of a selected geographical area.

For example, the project can be demonstrated using a particular campus, locality or nearby road network. Roads and intersections will be represented on the map. Each detected road defect will be associated with a road/location and displayed using a marker.

A simple condition representation can be:

- Green: Good road condition

- Yellow: Moderate road condition

- Red: Poor/Severe road condition

When the user selects a marker, the system can display:

Defect ID

Type: Pothole

Severity: High

Road: Road A

Status: Pending

Photo: View Image

The map can also highlight roads requiring urgent maintenance. OpenStreetMap will provide the real-world 2D road-map background, while the application will place its own colour-coded defect markers on top of it. For the academic prototype, after uploading a photograph, the user can click the exact defect location on the map; the application will capture the selected latitude and longitude and store them with the defect record and image reference. This avoids

requiring dedicated GPS hardware. Each marker can be linked to a Defect ID so that clicking it displays the defect details and corresponding photograph. Proper OpenStreetMap attribution and tile-usage requirements will be followed. In a future version, GPS coordinates can be automatically attached to photographs.

8. APPLICATION OF DATA STRUCTURES AND ALGORITHMS

8.1 Queue

A Queue follows FIFO (First In, First Out). It can store incoming road images waiting to be processed.

8.2 Linked List

A Linked List can maintain the history of detected road defects. Each node may store defect ID, type, severity, location, image reference and repair status.

8.3 Priority Queue / Heap

A Priority Queue organizes repair tasks based on severity, allowing urgent defects to be considered first.

8.4 Graph

The road network is represented using a weighted Graph.

- Vertices/Nodes = Intersections or locations
- Edges = Roads
- Edge Weight = Distance or travel cost

8.5 Dijkstra / A*

Dijkstra's algorithm or A* can find a shortest or low-cost route between the maintenance depot and defect locations.

8.6 Searching and Sorting

Searching can find a defect by ID, type, road or status. Sorting can arrange defects according to severity, size or priority.

9. APPLICATION OF OBJECT-ORIENTED PROGRAMMING

9.1 Abstraction

A general RoadDefect class can define important operations such as severity calculation and displaying details without exposing unnecessary implementation details.

9.2 Encapsulation

Attributes such as defect type, severity, location and repair status are kept inside the class and accessed through suitable member functions.

9.3 Inheritance

A common base class RoadDefect can have derived classes:

RoadDefect

|-- Pothole

|-- Crack

|-- DamagedEdge

9.4 Polymorphism

A function such as calculateSeverity() can behave differently for different defect classes.

10. SYSTEM ARCHITECTURE

The system consists of:

- Image Input Module

- Image Processing Module

- Defect Detection Module

- Severity Analysis Module

- OOP Defect Management Module

- DSA Management Module

- Map and Road Graph Module

- Route Planning Module

- GUI Dashboard

Overall:

Road Photos -> Image Processing -> Defect Detection -> Severity + Location -> DSA/OOP -> Repair Priority -> Road Graph -> Dijkstra/A* -> GUI/Map

11. TECHNOLOGY STACK

- C for DSA implementation.

- C++ for OOP and application logic.

- OpenCV or equivalent image-processing library.

- OpenStreetMap for map data/tiles and Leaflet for interactive map display.

- GTK or another suitable desktop GUI framework.

- Dijkstra, A*, searching and sorting algorithms.

- VS Code, GCC, G++ and Git/GitHub.

12. METHODOLOGY

week 1 – Requirement Analysis: Identify requirements, input format, defect categories, map requirements and expected outputs.

week 2 – Image Collection: Collect road photographs containing potholes, cracks and other visible defects.

week 3 – Image Processing: Apply preprocessing operations.

week 4 – Defect Detection: Identify candidate damaged regions.

week 5 – OOP Implementation: Create RoadDefect and derived classes.

week 6 – DSA Implementation: Implement Queue, Linked List, Priority Queue and Graph.

week 7 – Map Integration: Create a 2D representation of the selected area and associate defect locations with roads.

week 8 – Route Planning: Use Dijkstra or A* to calculate a suitable maintenance route.

week 9 – GUI Integration: Combine image display, defect records, map, priority information and route output.

week 10 – Testing: Test image processing, classification, severity, DSA operations, map marking and route calculation.

13. EXPECTED OUTCOME

The final system is expected to provide a working prototype in which a user can upload road photographs and obtain structured information about road conditions.

The application will be able to:

- Process road photographs.

- Identify visible road defects.

- Classify defect types.

- Estimate severity.

- Associate defects with road locations.

- Mark road conditions on a 2D map.

- Maintain defect history.

- Prioritize repair tasks.

- Represent the road network as a graph.

- Suggest suitable maintenance routes.

- Display results through a GUI.

14. PROJECT DELIVERABLES

- SmartRoad application.

- Image-processing and defect-detection module.

- DSA modules in C.

- OOP modules in C++.

- 2D road-condition map.

- Road-defect data management.

- Priority-based repair management.

- Graph-based route-planning module.

- GUI dashboard.

- Sample road-image dataset.

- Testing results.

- Project report and presentation.

- Final project demonstration.

15. ASSUMPTIONS

- Road defects are sufficiently visible in input photographs.

- Images have reasonable resolution and lighting.

- The prototype initially supports a predefined set of defect categories.

- Severity can be estimated using predefined rules or image-analysis features.

- Location can be manually assigned or stored with the image.

- The selected map area is predefined for the prototype, and OpenStreetMap is used as the map background.

- The project is an academic prototype and does not replace professional civil-engineering inspection.

16. LIMITATIONS

Image-based defect detection may be affected by poor lighting, rain or water, shadows, blurred images, low-resolution photographs, different camera angles, occluded defects and similar-looking road textures. Detection accuracy will depend on the quality and diversity of the dataset and the selected detection technique.

17. FUTURE SCOPE

- Integrate AI/deep-learning models for improved pothole and crack detection.

- Automatically obtain GPS coordinates from the device or image metadata.

- Integrate a real digital map with road geometry and location markers.

- Store repeated observations and monitor road deterioration over time.

- Generate optimized routes for multiple high-priority repair locations.

- Add database support for large numbers of images and defect records.

- Add cloud-based access for maintenance teams.

- Use historical data for predictive road maintenance.

18. CONCLUSION

SmartRoad is an image-based road-condition monitoring and maintenance-prioritization system that uses road photographs instead of dedicated physical sensors. It combines image processing with DSA, OOP, graph algorithms and GUI development.

The system detects visible road defects, estimates their severity, associates them with locations and displays them on a 2D map. DSA structures provide efficient organization and prioritization of defects, while OOP provides a modular representation of different road-defect types. Graph algorithms enable maintenance-route planning.

The proposed project provides a practical implementation of computer-science concepts in a real-world infrastructure problem and can later be extended with AI-based detection, GPS, real maps, databases and predictive maintenance.

19. REFERENCES

- T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, MIT Press.

- E. Balagurusamy, Object Oriented Programming with C++.

- OpenCV Documentation – Image Processing and Computer Vision.

- Research literature on automated pothole and crack detection using computer vision.

- Research literature on graph-based shortest-path algorithms and road-network optimization.